

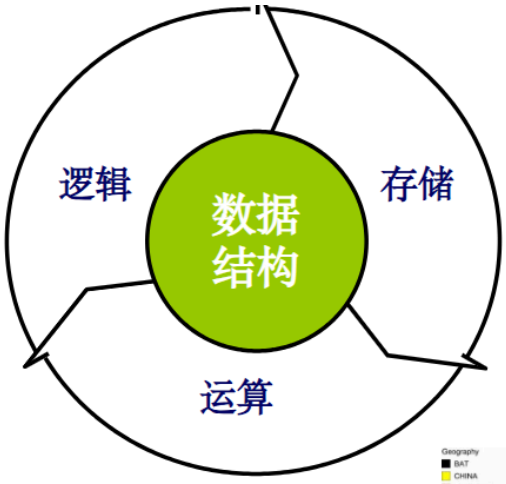
数算期中整理

Created @October 22, 2025 8:44 PM

第一章 概论

真正的数算从数据结构的定义和概念开始（

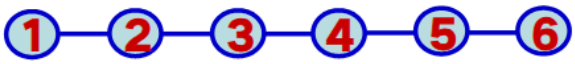
- **结构: 实体 + 关系**
- 数据结构:
  - 按照逻辑关系组织起来的一批数据,
  - 按一定的存储方法把它存储在计算机中
  - 在这些数据上定义了一个运算的集合

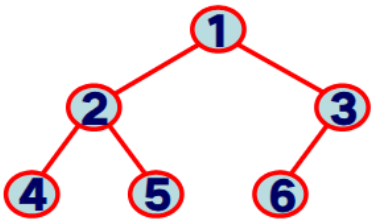


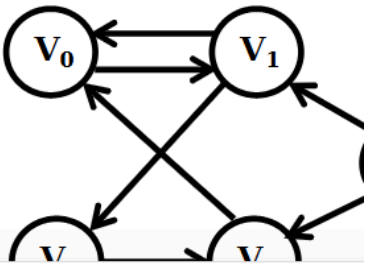
坑在哪？？？

- **线性结构**
  - **线性表（表，栈，队列，串等）**
- **非线性结构**
  - **树（二叉树，Huffman树，二叉检索树等）**
  - **图（有向图，无向图等）**
- **图 ⊇ 树 ⊇ 二叉树 ⊇ 线性表**

这里可能有坑!







抽象数据结构：ADT

<数据对象D，数据操作P>

先定义逻辑结构，再定义运算

逻辑结构：数据对象及其关系

运算：数据操作

存储结构：逻辑结构的存储映射？（王老师ppt没有给严格定义x

参考之前：

存储结构

- **数据的存储结构（Storage Structure）**
  - 也称物理结构（Physical Structure）
  - 是逻辑结构在计算机中的物理存储表示
- **计算机主存储器**
  - **空间相邻**：其存储空间提供了一种具有非负整数地址编码的、存储空间相邻的单元集合，其基本的存储单元是字节
  - **随机访问**：计算机的指令具有按地址随机访问存储空间内任意单元的能力，访问不同地址所需的访问时间基本相同

存储结构(Con.)

- 存储结构建立一种逻辑结构到物理结构的映射
  - 逻辑结点映射：对于每一个结点$j \in K$都对应一个唯一的存储区域$c \in M$。
  - 逻辑关系映射：每一关系元组$(j_1, j_2) \in r$（其中$j_1, j_2 \in K$是结点），亦即j_1, j_2的逻辑关系映射为存储单元的地址顺序关系（或指针地址指向关系）
- 四种基本存储映射方法
  - 顺序、链接、索引、散列

除了数据结构的相关概念之外，算法分析是第一章的重点：

大O表示：卡上界

单位时间：

- 简单布尔或算术运算
- 简单I/O
  - 指函数的输入/输出
    例如，从数组读数据等操作
  - 不包括键盘文件等I/O
- 函数返回

运算法则：

- **加法规则：** $f_1(n)+f_2(n)=O(\max(f_1(n), f_2(n)))$
  - **顺序结构，if** 结构，**switch**结构
- **乘法规则：** $f_1(n) \cdot f_2(n) = O(f_1(n) \cdot f_2(n))$
  - **for, while, do-while**结构

Ω表示：

下界

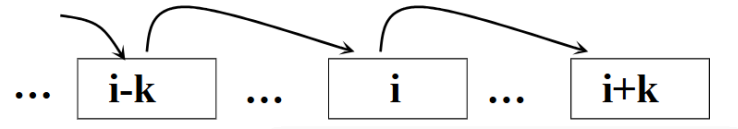
θ表示：
上下界相同

对于算法复杂度的两个举例：

海豚算法：时间O(n)， 空间O(1)

设计一个算法，将数组 A(0..n-1) 中的元素循环右移 k 位，假设原数组序列为 $a_0, a_1, \ldots, a_{n-2}, a_{n-1}$；移动后的序列为 $a_{n-k}, a_{n-k+1}, \ldots, a_0, a_1, \ldots, a_{n-k-1}$。要求只用一个元素大小的附加存储，元素移动或交换次数与 n 线性相关。例如，n=10, k=3

原始数组： 0 1 2 3 4 5 6 7 8 9
右移后的： 7 8 9 0 1 2 3 4 5 6



海豚算法：时间O(n)，空间O(1)

```
void Dolphin(int *Array, int n, int k) {
  int i,j,d;   int X;
  k = k % n;  if (k == 0) return;      // 边界
  d = gcd(n,k);                        // d为n和k的最大公约数
  for (i=1; i<=d; i++) {
      X = Array[n-i];                  // 先保存最尾部的元素
      // 步长为k循环，视数组为环状。
      for (j=n-i-k; j!=n-i; j=(n+j-k)%n)
          Array[(j+k)%n] = Array[j];  // 移动
      Array[(n-i+k)%n] = X;            // 补回x
  }
}
```

算法复杂度?
https://oi-wiki.org/math/number-theo

0 1 2 3 4 5 6 7 8 9
6 7 0 1 2 3 4 5 8 9

置逆算法：

置逆算法

```
void Reverse(int *Array, int n, int k)  {
        for (int i=0; i<n/2; i++)
                swap(Array, i, n-i-1);
        for (i=0; i<k/2; i++)
            swap(Array, i, k-i-1);
        for (i=0; i<(n-k)/2; i++)
                swap(Array, k+i, n-i-1);
}
```

- 时间代价为O(n)，空间代价为O(1)

补一个概念的问题：

算法的渐进分析

$$f(n) = n^2 + 100n + log_{10}n + 1000$$

- 算法分析：估计当数据规模n逐步增大时，（时间或者空间）资源开销f(n)的增长规模
- 从数量级大小的比较来考虑，当n增大到一定值时，资源开销的计算公式中影响最大的就是n的幂次最高的项，其他的常数项和低幂次项均可忽略不计

第二章 线性表

首先是对线性表的概念和分类：

线性结构：

存储上可以分顺序表和链式表(进一步单链、双链、循环链)

操作上可以“特化”出一端操作的栈，和两端操作的队列

接下来对于各种具体的结构和操作，我们要应用算法的分析：

顺序表：
插入删除：n/2, n-1/2 O(n)

按位置查找：O（1）（顺序存放）；按内容找（O（n））

链表：

首先区分带不带头节点

带头节点的话就是多一个存储节点空间，且空表的head！=NULL，反之不带就是head==NULL

操作（以“插入”为例）的区别

1. 不带头节点的链表

- **插入到空链表**：需直接将头指针、`curr`、`tail`指向新节点（如 PPT 中“空链表？→Yes”的分支）。
- **插入到链表头部（第一个有效节点前）**：需特殊处理头指针，让新节点成为新的头节点（如 PPT 中“curr==head？→Yes”的分支）。
- **插入到中间 / 尾部**：需额外追踪前驱节点（如 PPT 中的`temp`），且要区分“头节点”这一特殊情况，逻辑分支多。

2. 带头节点的链表(方便处理空或头部)

- 无论插入到**头部、中间还是尾部**，都可以通过**头节点**统一处理“前驱节点”的查找，不需要区分“空表”或“头节点插入”的特殊情况。只用看插入位置是不是尾（tail)
- 头节点存在时，只需找到`curr`的前驱（`fence`），再调整`fence`的`next`和新节点的`next`即可，逻辑更统一。

（带头节点：

链表插入

```
ListNode* Insert(int i,T value){
    ListNode *p,*q;
    q=new ListNode;
    p=setPos(i-1);
    if(p==NULL)return false;
```

```
    q→data=value;
    q→next=p→next;
    p→next=q;
    if(q→next==NULL)
        tail=q;
    return true;
}
```

链表删除

```
template<class T>
bool lnkList<T>::delete(const int i){
    Link<T>*p,*d;
    if((p=setPos(i-1))==NULL||p==tail){
        cout<<"非法删除点"<<endl;
        return false;
    }
    d=p→next;
    if(d=tail){
        tail=p;
        p→next=NULL;
        delete d;
    }
    else{
        p→next=d→next;
        delete d;

    }
    return true;
}
```

注意本身的话链式结构应该很方便插入删除，但是检索需要O（n)

单链表上运算的分析

1. 对一个结点操作，必先找到它，即用一个指针指向它
2. 找单链表中任一结点，**都必须从第一个点开始**

```
p = head;
while (没有到达) p = p->next;
```

- 单链表的时间复杂度 $O(n)$
  - 定位：$O(n)$
  - 插入：$O(n)+O(1)$
  - 删除：$O(n)+O(1)$

第三章 栈与队列

一个经典问题：

出栈可能性：

我们可以通过递归关系来推导。设 f(n) 表示 n 个元素的出栈序列数。显然：

当 n = 0 时，只有一个空序列,所以 f(0) = 1。

当 n ≥ 1 时，考虑第一个出栈的元素是第 k 个元素（其中 1 ≤ k ≤ n）。这意味着：

元素 1 到 k−1 必须在第 k 个元素出栈之前已经入栈并出栈，这些元素的出栈序列有 f(k−1) 种可能。

第 k 个元素出栈后，元素 k+1 到 n 必须入栈并出栈，这些元素的出栈序列有 f(n−k) 种可能。

因此，对于每个 k，出栈序列数为 f(k−1)·f(n−k)。求和得到：

f(n) = ∑(k=1 to n) f(k−1)f(n−k)

这正是卡塔兰数的递归关系。解这个递归关系可得 f(n) = T_n，

也即**(1/n+1)*C（2n，n)**

栈同样有顺序和链式的存储之分，

先来看抽象数据类型

```
template <class T> class arrStack : public Stack <T> {
private:                            // 栈的顺序存储
    int mSize;                      // 栈中最多可存放的元素个数
    int top;                        // 栈顶位置，应小于mSize
    T *st;                          // 存放栈元素的数组
public:                             // 栈的运算的顺序实现
    arrStack(int size) {            // 创建一个给定长度的顺序栈实例
        mSize = size; top = -1; st = new T[mSize];
    }
    arrStack() {                    // 创建一个顺序栈的实例
        top = -1;
    }
    ~arrStack() { delete [] st; }
    void clear() { top = -1; }  // 清空栈
}
```

顺序栈溢出：（定最大长度）

- **上溢 (Overflow)**
  - 当栈中已经有maxsize个元素时，如果再做进栈运算，所产生的现象
- **下溢 (Underflow)**
  - 对空栈进行出栈运算时所产生的现象

压栈：

```
bool arrStack<T>::push(const T item) {
    if (top == mSize-1) {       // 栈已满
        cout << "栈满溢出" << endl;
        return false;
    } else {          // 新元素入栈并修改栈顶指针
        st[++top] = item;
        return true;
    }
}
```

弹栈：

```
bool arrStack<T>::push(const T item) {
    if (top == mSize-1) {       // 栈已满
        cout << "栈满溢出" << endl;
        return false;
    } else {          // 新元素入栈并修改栈顶指针
        st[++top] = item;
        return true;
    }
}
```

链式栈用单链表存储，从栈顶向下长

```
template <class T> class lnkStack : public Stack <T> {
private:                                  // 栈的链式存储
    Link<T>* top;                         // 指向栈顶的指针
    int size;                             // 存放元素的个数
public:                                   // 栈运算的链式实现
    lnkStack(int defSize) {               // 构造函数
        top = NULL; size = 0;
    }
    ~lnkStack() {                         // 析构函数
        clear();
    }
}
```

入栈和顺序的相比要记得更新一下link

```
// 入栈操作的链式实现
bool lnksStack<T>:: push(const T item) {
    Link<T>* tmp = new Link<T>(item, top);
    top = tmp;
    size++;
    return true;
}
Link(const T info, Link* nextValue)  {// 具有两个参数的Link构造函数
        data = info;
        next = nextValue;
}
```

出栈判断空的标准也变成了size是0而不是top是-1

```
// 出栈操作的链式实现
bool lnkStack<T>:: pop(T& item) {
    Link <T> *tmp;
    if (size == 0) {
        cout << "栈为空，不能执行出栈操作"<< endl;
        return false;
    }
    item = top->data;
    tmp = top->next;
    delete top;
    top = tmp;
    size--;
    return true;
}
```

注意栈的访问限制端口，所以插入删除的复杂度就都是O(1)，顺序栈和链式栈只在空间效率上有差距，也是自然的

- **时间效率**
  - 所有操作都只需常数时间
  - 顺序栈和链式栈在时间效率上难分伯仲
- **空间效率**
  - 顺序栈须说明一个固定的长度
  - 链式栈的长度可变，但增加结构性开销

一个不太“栈”的角度：读取内部元素：本质上还是顺序存储和链式存储的区别

- 实际应用中，顺序栈比链式栈用得更广泛
  - 顺序栈容易根据栈顶位置，进行相对位移，快速定位并读取栈的内部元素
  - 顺序栈读取内部元素的时间为$O(1)$，而链式栈则需要沿着指针链游走，显然慢些，读取第k个元素需要时间为$\mathbf{O}(k)$

- 一般来说，栈不允许“读取内部元素”，只能在栈顶操作

特别注意PPT种强调了STL的栈和之前说的不同，pop不再返回栈顶的值

STL 将`top`和`pop`操作分开，主要基于**异常安全、设计原则、使用场景灵活性**这几方面的考量

栈的链表实现对比

- **是否可以用双链表**：可以。
- **哪个更好（单链表更优）**
  - **空间效率**：单链表节点仅需存储数据和后继指针，双链表需额外存储前驱指针，空间开销更大。
  - **操作效率**：栈的核心操作（push、pop）仅需在栈顶进行，单链表通过头节点即可实现 O (1) 的插入和删除；双链表的前驱指针在此场景下无实用价值，无法提升操作效率。
  -

前缀表达式（波兰式）的性质与求值过程

一、性质

- 运算符位于对应操作数的**前面**，如 “+ 3 4” 表示 “3+4”。
- 无歧义性，不需要括号即可明确运算顺序。
- 可通过栈结构高效求值。

二、求值过程

1. **扫描方向**：从右到左扫描表达式。
2. **操作逻辑**：
    - 遇到**操作数**，将其压入栈。
    - 遇到**运算符**，从栈中弹出两个操作数（先弹出的为右操作数，后弹出的为左操作数），用该运算符计算，再将结果压入栈。
3. **最终结果**：扫描结束后，栈中仅剩余一个元素，即为表达式的结果。

示例：对前缀表达式 “+ * 2 3 4” 求值

- 扫描顺序：4 → 3 → 2 → * → +
- 操作过程：
    - 压入 4、3、2；
    - 遇到 “*”，弹出 2、3，计算 2×3=6，压入 6；
    - 遇到 “+”，弹出 6、4，计算 6+4=10；
    - 最终结果为 10。

栈的核心应用

1. 关于表达式

后缀自然匹配栈结构

后缀表达式求值

- 循环：依次顺序读入表达式的符号序列（假设以＝作为输入序列的结束），并根据读入的元素符号逐一分析
  1. 当遇到的是一个操作数，则压入栈顶
  2. 当遇到的是一个运算符，就从栈中两次取出栈顶，按照运算符对这两个操作数进行计算。然后将计算结果压入栈顶
- 如此继续，直到遇到符号＝，这时栈顶的值就是输入表达式的值

前缀要算就是从右到左扫描就行

中缀的话麻烦一点，要搞两个栈，一个存数，一个符号

2. 递归转换

递归的理解：
Hanoi塔的经典问题

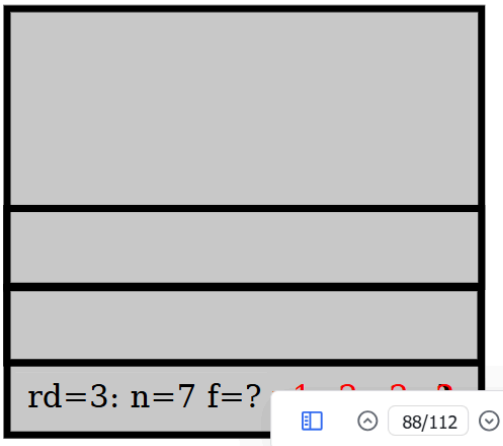
```
void hanoi(int n, char X, char Y, char Z) {
    if (n <= 1)
        move(X,Z);
    else { // X上最大的不动，其他 n - 1个环槃移到Y
        hanoi(n-1,X,Z,Y);
        move(X,Z);       // 移动最大环槃到Z，放好
        hanoi(n-1,Y,X,Z); // 把 Y上的n - 1个环槃移到Z
    }
}
void move(char X, char Y)   //  把柱X的顶部环槃移到柱Y
{
  cout  << "move"  <<  X  << "to" << Y  << endl;
}
```

递归的本质就是函数的调用，对应了堆栈的进退

用栈模拟递归调用过程

- 后调用，先返回（LIFO），所以用栈

```
void exmp(int n, int& f) {
    int u1, u2;
    if (n<2) f = n+1;
    else {
        exmp((int)(n/2), u1);
        exmp((int)(n/4), u2);
        f = u1*u2;
    }
}
```



机械做法（像编译器）

就是搞出一个工作栈，每次递归调用就压栈，返回就弹栈（真不是ICS？x）

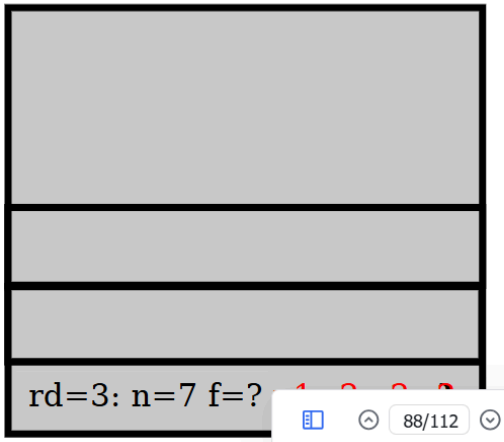
第三章

栈与队列

3.1.3 递归转非递归

用栈模拟递归调用过程

- 后调用，先返回（LIFO），所以用栈

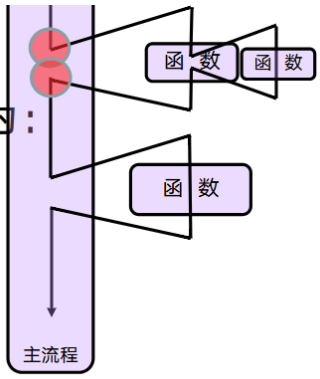



```
void exmp(int n, int& f) {
   int u1, u2;
   if (n<2) f = n+1;
   else {
      exmp((int)(n/2), u1);
      exmp((int)(n/4), u2);
      f = u1*u2;
   }
}
```

88/112

把调用语句改成一个goto

4. 替换第i (i=1,...,t)个递归规则



- 假设函数体中第i (i=1, ..., t)个递归调用语句为：recf(a1, a2, ...,am);
- 则用以下语句替换：

```
S.push(i, a1, ..., am);  // 实参进栈
goto label 0;
.....
label i:  x = S.top(); S.pop();
/* 退栈，然后根据需要，将x中某些值赋给栈顶的工作记录S.top () — 相当于把引用型参数的值回传给局部变量 */
```

递归出口也类似给一个goto（t+1)（假设共t个递归规则）

注意循环和嵌套需要去掉，然后应用4（有时需要把带返回值的函数改成void，返回值改成参数）

7. 改写循环和嵌套中的递归

- 对于循环中的递归，改写成等价的goto型循环
- 对于嵌套递归调用

例如，recf (... recf())

改为：

$exmp_1$ = recf ( );

$exmp_2$ = recf ($exmp_1$);

...

$exmp_k$ = recf ($exmp_{k-1}$)

然后应用规则 4 解决

队列

很多内容和栈对称，

一个比较特殊的是由于队列某种程度上要求两端操作，所以不适合顺序表存储，

我们想让队头队尾的“指针”可以移动

也因此，我们的一个目标就是解决“假溢出”，数组的头（严格说好像叫下标~~指针~~并不在0处，而尾~~指针~~rear到达size-1时，实现“取模的循环队列”也就是“环”

在实际中采取“环状”的实指存储结构

但假如我们有n个元素的一个环，我们发现0，1，2，。。。，n我们有n+1种不同的状态需要表示，这样空和满无法区分

我们通常采用牺牲一个元素位置的办法：

我们把存储n个元素的数组大小设为n+1,

我们规定front=rear为空，而(rear +1) % front == 0就是满的标准，注意这时我们的rear代指最后一个元素的下一个位置，是虚指，

而像下面的代码rear是最后一个元素指向的位置，算实指（rear自动看作我们所记的rear-1)

- **实指更优**：逻辑直观，空队列判断简单，是更常用的实现方式，易理解、易维护。
- **虚指的特点**：在某些边界场景（如循环队列的变种实现）中可简化操作，但因空队列判断较间接，整体易用性不如实指。

```
template <class Elem> class Aqueue : public Queue<Elem> {
private:
    int size;                    // 队列的最大容量
    int front;                   // 队首元素指针
    int rear;                    // 队尾元素指针
    Elem *listArray;             // 存储元素的数组
public:
    AQueue(int sz=DefaultListSize) {
        // 让存储元素的数组多预留一个空位
        size = sz+1;             // size数组长，sz队列最大长度
        rear = 0; front = 1;     // 也可以rear=-1; front=0
        listArray = new Elem[size];
    }
    ~AQueue() { delete [] listArray; }
    void clear() { front = rear+1; }
```

```
bool enqueue(const Elem& it) {
    if (((rear+2) % size) == front) return false;
                            // 还剩一个空位，就要报满
    rear = (rear+1) % size; // 因为实指，需要先移动到下一个空位
    listArray[rear] = it;
    return true;
}
bool dequeue(Elem& it) {
    if (length() == 0) return false;
                                // 队列为空
    it = listArray[front];      // 先出队，再移动front下标
    front = (front+1) % size;   // 环形增加
    return true;
}
```

链式队列，同样用单链表，从队头一路链到队尾

链式队列代码实现

```
bool enQueue(const T item) { //  item入队，插入队尾
    if (rear == NULL) {       // 空队列
        front = rear = new Link<T> (item, NULL);
    }
    else  {                   // 添加新的元素
        rear-> next = new Link<T> (item, NULL);
        rear = rear ->next;
    }
    size++;
    return true;
}
```

```
bool deQueue(T* item)  {          // 返回队头元素并从队列中删除
    Link<T> *tmp;
    if (size == 0)  {             // 队列为空，没有元素可出队
        cout << "队列为空" << endl;
        return false;
    }
    *item = front->data;
    tmp = front;
    front = front -> next;
    delete tmp;
    if (front == NULL)
        rear = NULL;
    size--;
    return true;
}
```

53/112

无论顺序还是链式，队列都不允许访问内部元素

队列的应用：

宽搜

农夫过河？（感觉建模之后关系也不大？，本质就是个搜索吧qwq

第四章 字符串

第一块主要是模式匹配

朴素匹配 O（m*n）

KMP：无回溯！

（1）next数组

不赘述（

优化就是看next(K) = K的话，可以把next（K）优化成next(next(K))

（2）next数组指示的就是我们到某一位不匹配之后往后移多少位（i- next(i))

insight是对于子串内部先有一个预处理，

所以O（m+n)

不同版本特征值？这个意义是啥（x

思考：不同版本特征值定义

j 位匹配错误，则 j=next[j]

$$next[j]=\begin{cases}-1, & \text{对于j}=0\\ \max\{k: 0<k<j \ \&\& \ P[0...k-1]=P[j-k...j-1]\}, & \text{如果k存在}\\ 0, & \text{否则}\end{cases}$$

j 位匹配错误，则 j=next[j-1]

$$next[j]=\begin{cases}0, & \text{对于j}=0\\ \max\{k: 0<k<j \ \&\& \ P[0...k]=P[j-k...j]\}, & \text{如果k存在}\\ 0, & \text{否则}\end{cases}$$

然后子串相关

操作类别	方法	描述
子串	substr ()	返回一个串的子串
拷贝/交换	swap ()	交换两个串的内容
	copy ()	将一个串拷贝到另一个串中
赋值	assign ()	把一个串、一个字符、一个子串赋值给另一个串中
	=	把一个串或一个字符赋值给另一个串中
插入/追加	insert()	在给定位置插入一个字符、多个字符或串
	append () / +=	将一个或多个字符、或串追加在另一个串后
拼接	+	通过将一个串放置在另一个串后面来构建新串
查询	find ()	找到并返回一个子序列的开始位置
替换/清除	replace ()	替换一个指定字符或一个串的字串
	clear ()	清除串中的所有字符
统计	size () / length()	返回串中字符的数目
	max_size ()	返回串允许的最大长度

重复子串等算法？

比如最大重复子串：

一、朴素算法的核心步骤

1. **枚举所有子串**：遍历字符串的所有可能子串（由起始位置 `i` 和结束位置 `j` 确定，$0 \le i < j < n$）。
2. **检查子串是否重复**：对每个子串 `S1 = str[i..j]`，在其后方（`k > j` 位置）枚举所有长度相同的子串 `S2 = str[k..k+(j-i)]`，判断 `S1` 和 `S2` 是否相等。
3. **记录最长重复子串**：遍历过程中更新 “最长重复子串” 及其下标。

二、时间复杂度 $O(n4)$ 的推导

- **步骤 1：枚举子串的数量**：字符串长度为 n 时，子串的数量为 $O(n^2)$（起始位置 `i` 有 n 种选择，结束位置 `j` 有 $n-i$ 种选择。
- **步骤 2：检查子串重复的成本**：
  - 对于每个子串 `S1`，其长度为 $l = j - i + 1$。需在后续字符串中枚举所有长度为 l 的子串 `S2`，可能的枚举次数为 $O(n)$（因为 `k` 的起始位置需从 `j+1` 开始，最多有 $n-l$ 个位置）。
  - 比较 `S1` 和 `S2` 是否相等的时间为 $O(l)$（需逐字符对比，最多对比 l 个字符）。
- **总时间复杂度**：将上述步骤的成本相乘，得到 $O(n2) \times O(n) \times O(l)$。由于 l 最大为 n，且需对所有可能的 l（$1 \le l \le n$）求和，最终复杂度为 $O(n4)$（可理解为 “子串对的数量 $O(n4)$ 乘以单次比较的时间 $O(1)$ 到 $O(n)$，最坏为 $O(n4)$”）。

三、空间复杂度 $O(n)$ 的推导

朴素算法仅需存储**当前子串的字符**（用于和其他子串比较），而子串的最大长度为 n，因此额外空间开销为 $O(n)$。

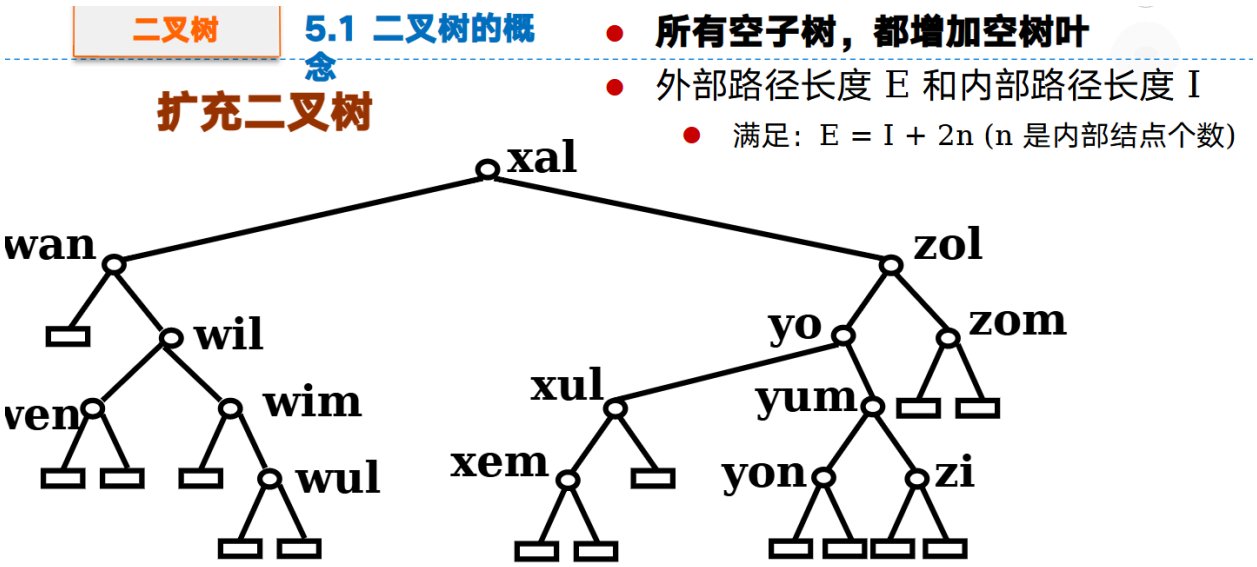
综上，朴素算法因需要暴力枚举所有子串对并逐字符比较，导致时间复杂度达到 $O(n4)$；而空间上仅需存储单个子串，故为 $O(n)$。

第五章 二叉树

~~感觉内容最丰富的一章，先略记（~~

1.首先是基本概念，注意一些数量关系

扩充二叉树：



2.遍历

深搜：

非递归算法

- [x] ~~非递归的伪代码这些还是要补看一下~~

用栈

前序：

```
BinaryTree<T>::PreOrderWithoutRecusion
(BinaryTreeNode<T>* root) {
```



```
using std::stack;                    // 使用STL中的stack
stack<BinaryTreeNode<T>* > aStack;
BinaryTreeNode<T>* pointer=root;
aStack.push(NULL);                   // 栈底监视哨
while(pointer) {                     // 或者!aStack.empty()
    Visit(pointer->value());         // 访问当前结点
    if (pointer->rightchild() != NULL)   // 右孩子入栈
        aStack.push(pointer->rightchild());
    if (pointer->leftchild() != NULL)
        pointer = pointer->leftchild();  //左路下降
    else  {                  // 左子树访问完毕，转向访问右子树
        pointer = aStack.top();
        aStack.pop();    // 栈顶元素退栈 }
} }
```

中序：

```
template<class T>  void
BinaryTree<T>::InOrderWithoutRecusion(BinaryTreeNode<T>
* root) {
    using std::stack;                    // 使用STL中的stack
    stack<BinaryTreeNode<T>* > aStack;
    BinaryTreeNode<T>* pointer = root;
    while (!aStack.empty() || pointer) {
       if (pointer ) {
         // Visit(pointer->value());     // 前序访问点
          aStack.push(pointer);          // 当前结点地址入栈
         // 当前链接结构指向左孩子
          pointer = pointer->leftchild();
```



```
    } //end if
   else {        //左子树访问完毕，转向访问右子树
       pointer = aStack.top();
       aStack.pop();                     //栈顶元素退栈
       Visit(pointer->value());          //访问当前结点
       //当前链接结构指向右孩子
       pointer=pointer->rightchild();
   } //end else
 } //end while
}
```

后序最完整

```
enum Tags{Left,Right};                    // 定义枚举类型标志位
template <class T>
class StackElement  {                     // 栈元素的定义
public:
    BinaryTreeNode<T>* pointer;     // 指向二叉树结点的指针
    Tags tag;                             // 标志位
};
template<class T>
void BinaryTree<T>::PostOrderWithoutRecursion(BinaryTreeNode<T>* root)
{
    using std::stack;                     // 使用STL的栈
    StackElement<T> element;
    stack<StackElement<T > > aStack;
    BinaryTreeNode<T>* pointer;
    pointer = root;
```


5.2 二叉树的抽象数据类型

```
while (!aStack.empty() || pointer) {
  while (pointer != NULL) {                          // 沿非空指针压栈，并左路下降
    element.pointer = pointer;  element.tag = Left;
    aStack.push(element);                  // 把标志位为Left的结点压入栈
    pointer = pointer->leftchild();
  }
  element = aStack.top();  aStack.pop();             // 获得栈顶元素，并退栈
  pointer = element.pointer;
  if (element.tag == Left) {               // 如果从左子树回来
    element.tag = Right; aStack.push(element); // 置标志位为Right
    pointer = pointer->rightchild();
  }
  else {                                   // 如果从右子树回来
    Visit(pointer->value());               // 访问当前结点
    pointer = NULL;                        // 置point指针为空，以继续弹栈
  }
}
}
```

二叉树遍历算法的时间代价分析

- 在各种遍历中，每个结点都被访问且只被访问一次，时间代价为O(n)
- 非递归保存入出栈（或队列）时间
  - 前序、中序，某些结点入/出栈一次，不超过O(n)
  - 后序，每个结点分别从左、右边各入/出一次， O(n)

宽搜：队列

存储：

二叉链表

空间开销分析

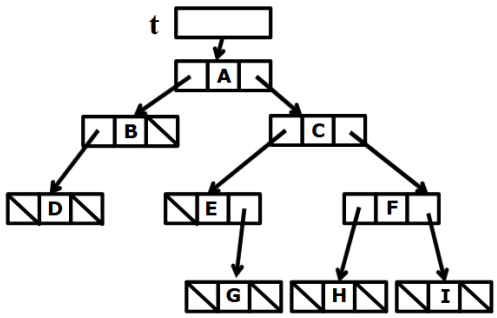
- 存储密度 α (≤1) 表示数据结构存储的效率

$$\alpha(\text{存储密度}) = \frac{\text{数据本身存储量}}{\text{整个结构占用的存储总量}}$$

- 结构性开销 γ = 1- α

空间开销分析

根据满二叉树定理：一半的指针是空的



- 每个结点存两个指针、一个数据域
  - 总空间 $(2p + d)n$
  - 结构性开销：$2pn$
  - 如果 $p = d$, 则结构性开销 $2p/(2p + d) = 2/3$

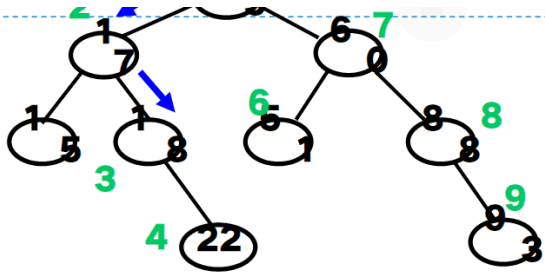
后面几个重要应用：

二叉搜索树

概念

二叉搜索树

- Binary Search Tree（**BST**）
  - **或者是一棵空树；**
  - **或者是具有下列性质的二叉树：**
    - **对于任何一个结点，设其值为K**
    - **则该结点的 左子树(若不空)的任意一个结点的值都 小于 K;**
    - **该结点的 右子树(若不空)的任意一个结点的值都 大于 K;**
    - **而且它的左右子树也分别为BST**
- 性质：中序遍历是正序的（由小到大的排列）



构建、插入、删除等操作

堆（最小）

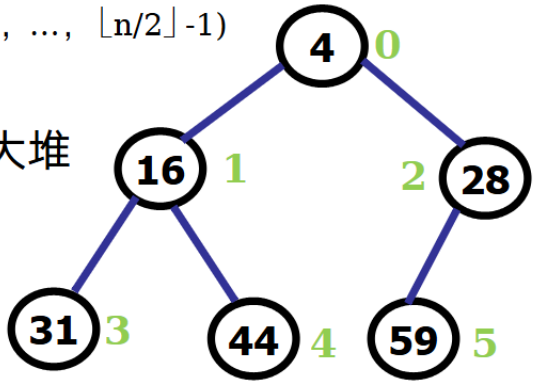
堆的定义及其实现

- 最小堆：最小堆是一个关键码序列

  $\{K_0, K_1, \ldots K_{n-1}\}$，它具有如下特性：

  - $K_i \le K_{2i+1}$　　$(i=0, 1, \ldots, \lfloor n/2 \rfloor -1)$
  - $K_i \le K_{2i+2}$

- 类似可以定义最大堆



siftup 和siftdown

建堆：

从底向上

用siftup,而且相同层顺序是从右至左

建堆效率分析

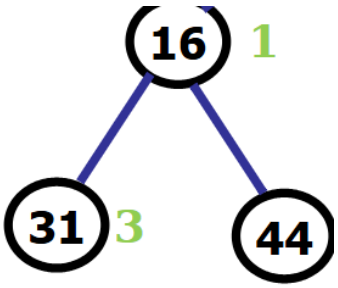
n 个结点的堆，高度 $d = \lfloor \log_2 n + 1 \rfloor$。
根为第 0 层，则第 i 层结点个数为 2^i，

考虑一个元素在堆中向下移动的距离。

- 大约一半的结点深度为 d-1，不移动（叶）。
- 四分之一的结点深度为 d-2，而它们至多能向下移动一层。
- 树中每向上一层，结点的数目为前一层的一半，而子树高度加一。因而元素移动的最大距离的总数为

$$\sum_{i=1}^{\log n} (i-1)\frac{n}{2^i} = O(n)$$

- 建堆算法时间代价为 O(n)



- 堆有 log n 层深
  - 插入结点、删除普通元素和删除最小元素的平均时间代价和最差时间代价都是 O(log n)

Huffman

前缀编码

每次选最小和次小，这样定序

第六章 树

首先，概念上，一定要注意区分，这个二叉树有严格的左子节点和右子节点

- 树 (tree) 是包括 n 个结点的有限集合 T（n ≥ 1）：
  - 有且仅有一个特定的结点，称为 **根 (root)**
  - 除根以外的其他结点被分成 m 个 (m ≥ 0) 不相交的有限集合 T_1，T_2，…，T_m，而每一个集合又都是树，称为 T 的 **子树 (subtree)**
  - 有向有序树：子树的相对次序是重要的
- 度为 2 的有序树并不是二叉树
  - 第一子结点被删除后
    第二子结点自然顶替成为第一



树的相关术语

- **度数**：一个结点的子树的个数
- **层数**：根为第 0 层
  - 其他结点的层数等于其父结点的层数加 1
- **深度**：层数最大的叶结点的层数
- **高度**：层数最大的叶结点的层数加 1

树的度指的就是节点的最大度

几个表示法

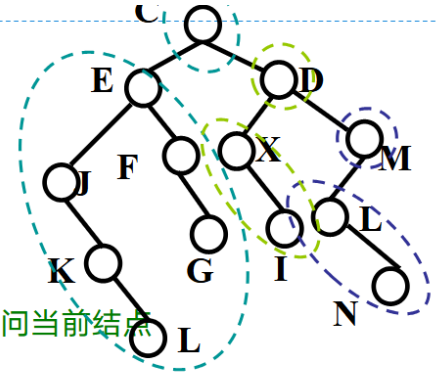
森林和二叉树等价转换：

左子右兄弟

然后遍历：

先根

先根深度优先遍历森林




```
template<class T>
void Tree<T>::RootFirstTraverse(
        TreeNode<T> * root)  {
    while (root != NULL) {
        Visit(root->Value());                    // 访问当前结点
        // 遍历第1棵树根的子树森林(树根除外)
        RootFirstTraverse(root->LeftMostChild());
        root = root->RightSibling();             // 遍历其他树
    }
}
```

后根

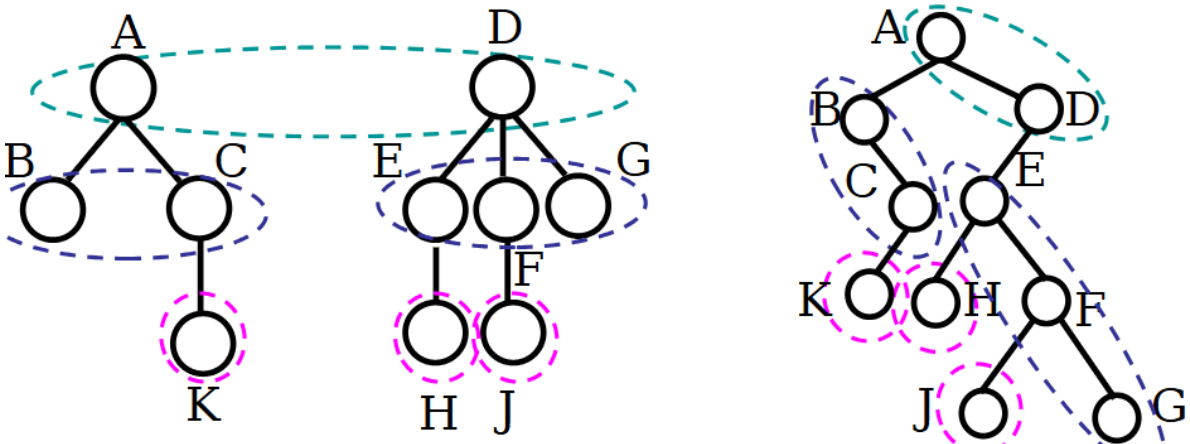
后根深度优先遍历森林

```
template<class T>
void Tree<T>::RootLastTraverse(
        TreeNode<T> * root)  {
    while (root != NULL) {
      // 遍历第一棵树根的子树森林
      RootLastTraverse(root->LeftMostChild());
      Visit(root->Value());            // 访问当前结点
      root = root->RightSibling();  // 遍历其他树
    }
}
```

29/89

广度优先：

广度优先遍历森林



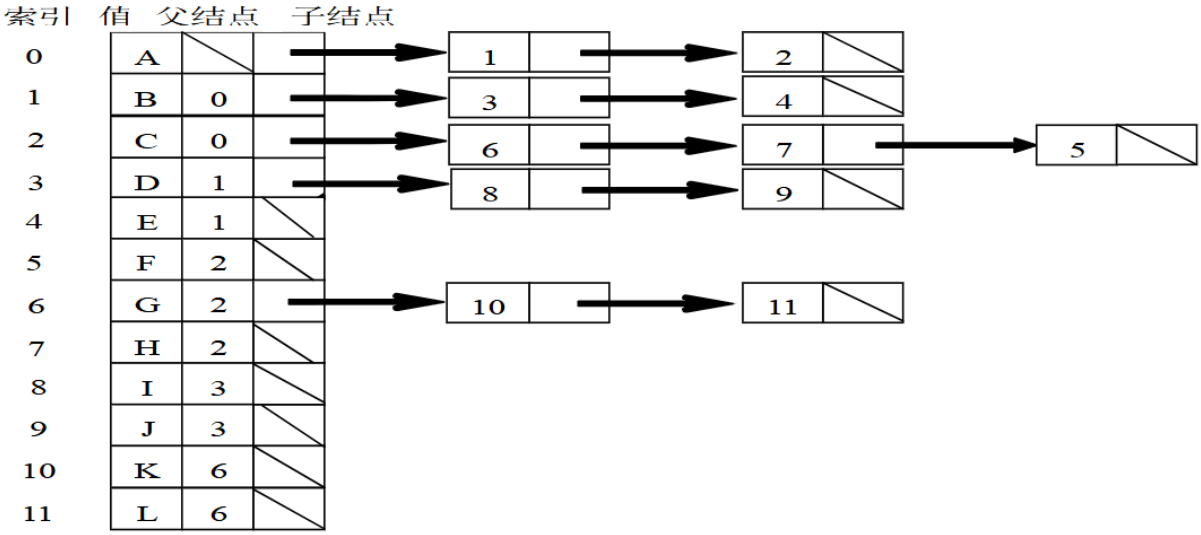
- 森林广度优先：A D B C E F G K H J
- **看二叉链存储结构的右斜线**

树的链式存储和表示方法

1. 子节点表

“子结点表”表示方法

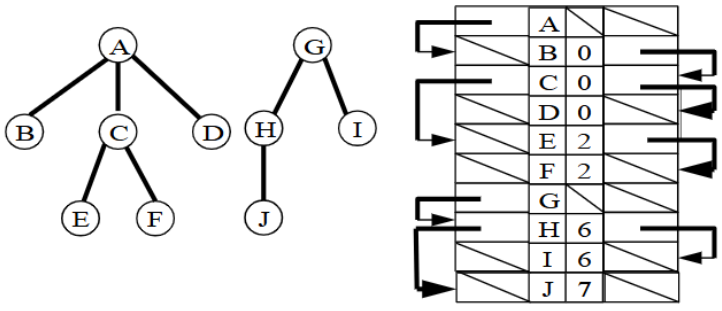
list of children，就是图的邻接表。



而到了数组的表示上，就是静态“左子右兄”

静态“左孩子/右兄弟”表示法

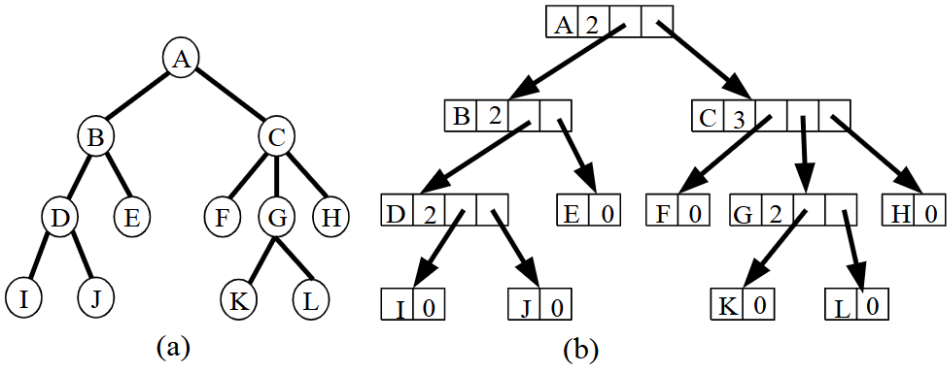
- 在数组中存储的“子结点表”



除此之外，还有“动态表示法”

动态表示法

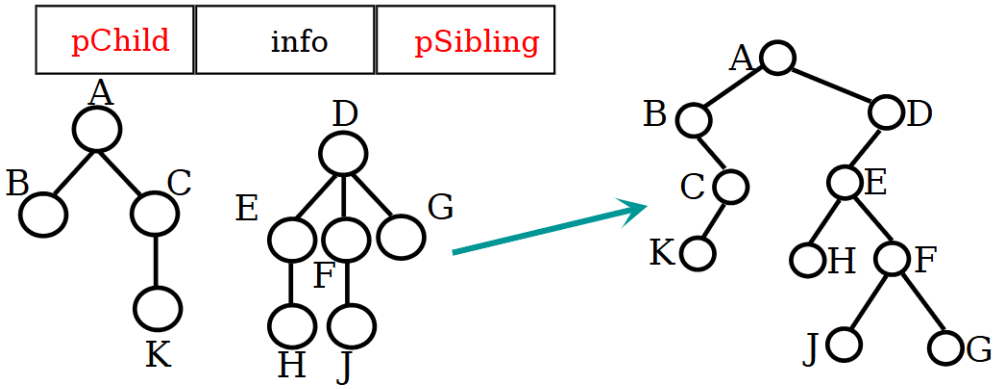
- 每个结点分配可变的存储空间
  - 子结点数目发生变化，需要重新分配存储空间



左子右兄“二叉链表”

动态“左子/右兄”二叉链表示法

- 左孩子在树中是结点的最左子结点，右子结点是结点原来的右侧兄弟结点
- 根的右链就是森林中每棵树的根结点



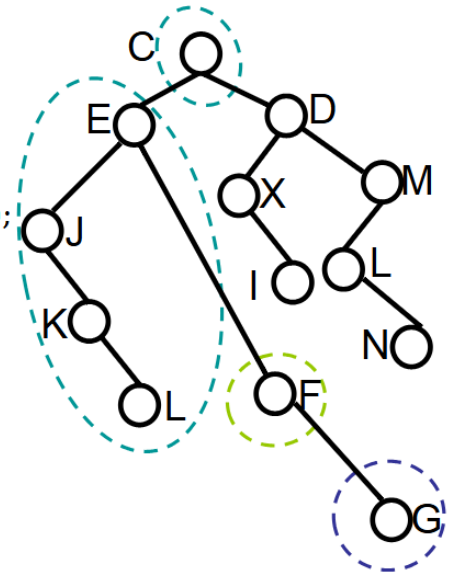
寻找当前节点父节点

（用队列）

P41

思考：下面的算法是否能遍历森林?

```
template <class T>
void Traverse(TreeNode <T> * rt) {
   if (rt==NULL)  return;
   Visit(rt);
   TreeNode * temp = rt-> LeftMostChild();
   while (temp != NULL) {
       Traverse(temp);
       temp = temp->RightSibling();
   }
}
```



函数 `Traverse(rt)` 采用**先根遍历**策略：

1. 若当前节点 `rt` 为空，直接返回；
2. 访问当前节点 `rt`；
3. 找到 `rt` 的**最左孩子** `temp`，递归遍历 `temp` 及其所有**右兄弟**（通过 `temp = temp->RightSibling()` 循环实现）。

对于**森林**（多棵树的集合），只需对每棵树的根节点依次调用该函数，即可完成遍历。

灵活应用遍历框架？森林镜面映射？（何意味。

删除以root为代表的森林所有结点：

（注意一个森林的root指的就是第一个树的根）

删除以subroot为根子树

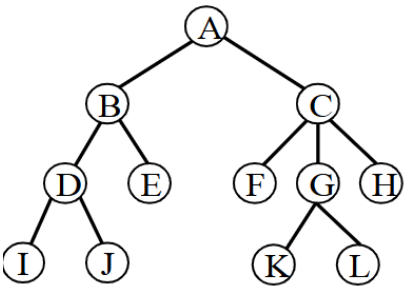
删除以subroot为根的子树

```
template<class T>
void Tree<T>::DeleteSubTree(TreeNode<T> *subroot)  {
    if (subroot == NULL) return;// 若待删除的子树为空则返回
    TreeNode<T> *pointer = Parent (subroot); // 找subroot的父结点
    if (pointer == NULL) {// subroot没有父，则是某个树根
      pointer =  root;
      while (pointer->RightSibling() != subroot)// 顺右链找左邻树根
        pointer = pointer->RightSibling();
      pointer->setSibling(subroot->RightSibling()); // 前后挂接，脱链
    }
    else if (pointer->LeftMostChild() == subroot) // subroot为最左子
      pointer->setChild(subroot->RightSibling()); // 挂新的最左
    else {// subroot有左兄弟的情况
      pointer = pointer->LeftMostChild();   // 下降到最左兄弟
      while (pointer->RightSibling() != subroot)// 顺右链找左邻兄弟
        pointer = pointer->RightSibling();
      pointer->setSibling(subroot->RightSibling()); // 前后挂接，脱链
    }
    subroot->setSibling(NULL);// 非常重要，丢了会出错
    DestroyNodes(subroot);  // 删除以subroot代表的子森林的所有结点
```

最后的链式表示：父指针表示法；以及其引出的并查集！

父指针表示法

- 只需要知道父结点的应用
- 只需要保存一个指向其父结点的指针域，称为 父指针(parent pointer)表示法
- 用数组存储树结点，同时在每个结点中附设一个指针指示其父结点的位置



0	1	2	3	4	5	6	7	8	9	10	11
A	B	C	D	E	F	G	H	I	J	K	L
＼	0	0	1	1	2	2	2	3	3	6	6

特点：

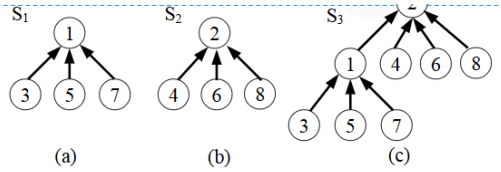
父指针表示法：算法

- 查询结点的根
  - 从一个结点出发找出一条向上延伸到达根的祖先路径
    - O(k)，k为树高
- 判断两个结点是否在同一棵树
  - 两个结点根结点相同，它们一定在同一棵树中
  - 如果其根结点不同，那么两个结点就不在同一棵树中

这样自然看到适合解决"等价类"问题，引出并查集！

union和find操作好用

用树来表示等价类的并查



用一棵树代表一个集合

- 集合用父结点代替
- 若两个结点在同一棵树中，则它们处于同一个集合

树的实现

- 存储在静态指针数组中
- 结点中仅需保存父指针信息

find自然就是看两个节点的根节点是否相同

union操作的trick：看两个树的“大小”（以此估计“深度”）这样把小的根节点挂到大的根上，希望最长的深度不增加；节省find时间

（加权合并）

路径压缩：在find过程中，把每次的父节点挂到根节点，这样能压缩路径长度

路径压缩使Find开销接近于常数

- 权重 + 路径压缩
- 对n个结点进行n次Find操作的开销为$O(n\alpha(n))$，约为$\Theta(n\log^* n)$
  - $\alpha(n)$是单变量Ackermann函数的逆，它是一个增长速度比logn慢得多但又不是常数的函数
  - $\log^* n$ 是在 $n = \log n \le 1$ 之前要进行的对 n 取对数操作的次数
  - $\log^* 65536 = 4$（4次log操作）
- Find至多需要一系列n个Find操作的开销非常接近于$\Theta(n)$
  - 在实际应用中，$\alpha(n)$往往小于4

思考题：(*没仔细看)

一、可否使用动态指针方式实现父指针表示法

可以。父指针表示法的核心是**每个节点存储指向其父节点的引用**，动态指针（如 C/C++ 的指针、Java 的对象引用）可直接实现这种引用关系，支持树结构的动态构建与调整（如并查集的树型组织、多叉树的父子关联）。

- 优势：节点父子关系可动态修改，内存管理灵活；
- 劣势：静态场景下效率不如数组索引方式，手动管理内存的语言需注意空指针与内存泄漏问题。

二、并查集权重（按秩合并）与路径压缩优化的异同及优劣

维度	权重（按秩合并）	路径压缩
核心思想	合并时将秩小的树挂到秩大的树根下，控制树高	查找时将路径节点直接指向根，扁平化结构
生效阶段	合并操作时	查找操作时
相同点	均为并查集优化手段，目标是降低树高、减少时间复杂度	
不同点	聚焦合并阶段的树高控制	聚焦查找阶段的路径扁平化

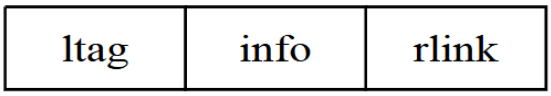
维度	权重（按秩合并）	路径压缩
优势	合并后树高增长缓慢，稳定性强	查找效率接近常数，多次查找优化显著
劣势	单独使用时查找路径仍可能较长	会破坏秩的准确性，若需维护树高信息则受限

总结：两者常结合使用（路径压缩 + 按秩合并），使并查集时间复杂度接近线性，是大部分场景下的最优策略。

顺序存储：注意这里所有tag都是有是0，没有是1

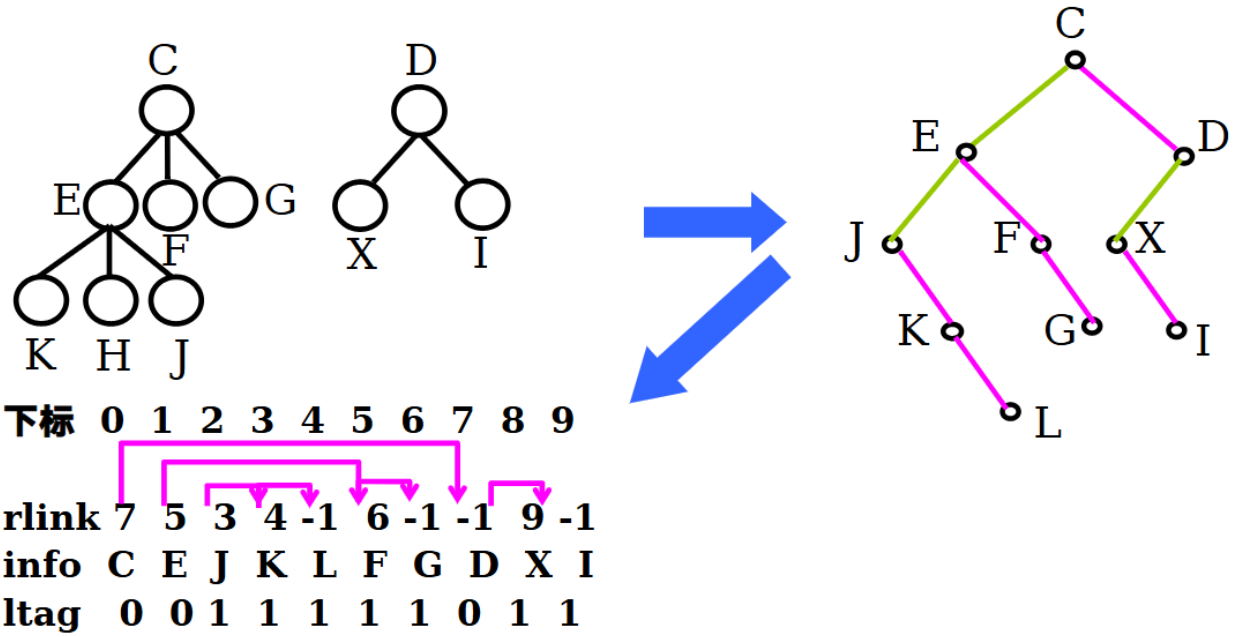
1.带右链先根

- 结点按先根次序顺序连续存储



  - info：结点的数据
  - rlink：右指针
    - 指向结点的下一个兄弟、即对应的二叉树中结点的右子结点
  - ltag：标记
    - 树结点没有子结点，即二叉树结点没有左子结点，ltag为 1
    - 否则为0

带右链的先根次序表示法



2.双标记先根

我们发现因为先根给出顺序，所以rlink作为int有些浪费，可以改成rtag

带双标记的先根次序表示

- 带右链的先根次序表示”中rlink也有冗余，可以把rlink指针替换为一个标志位rtag，成为“带双标记的先根次序表示”。其中，每个结点包括结点本身数据，以及两个标志位ltag和rtag，其结点的形式为：

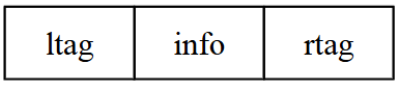
ltag	info	rtag

由结点的先根次序以及ltag、rtag两个标志位，就可以确定树“左孩子/右兄弟”链表中结点的llink和rlink值。其中llink的确定与带右链的先根次序表示法相同。

3.双标记层次次序

带双标记的层次次序表示法

- 结点按 层次次序顺序 存储在连续存储单元



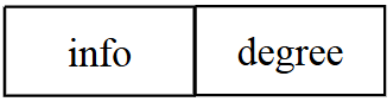
  - info是结点的数据
  - ltag是一个一位的左标记，当结点没有子节点，即对应的二叉树中结点没有左子结点时， ltag为1，否则为0
  - rtag是一个一位的右标记，当结点没有下一个兄弟，即对应的二叉树中结点没有右子结点时， rtag为1，否则为0

只是顺序变了，差不多

4.带度数后根

带度数的后根次序表示

- 在带度数的后根次序表示中，结点按后根次序顺序存储在一片连续的存储单元中，结点的形式为



- 其中info是结点的数据，degree是结点的度数

也是逐步从下往上还原就行

思考*：

一、信息冗余问题

森林的顺序存储若采用**类似完全二叉树的数组式存储**，对于非完全结构的树或森林，会出现大量**空节点占位**的情况。这些空节点不承载有效数据，却占用内存空间，导致**信息冗余**。例如，一棵节点分布稀疏的树，用顺序存储时，数组中大部分位置为 “空”，这部分空间即为冗余的存储开销。

二、树的其他顺序存储

1. 带度数的先根次序

- **原理**：先根次序（先访问根节点，再依次访问各子树）存储时，**每个节点附带记录自身的 “度数”（子节点的个数）**。
- **作用**：通过度数可明确 “当前节点的子树范围”。例如，若某节点度数为k，则其后续连续的k个节点即为它的子节点，从而在顺序存储中无歧义地还原树的结构。
- **适用场景**：适用于需要快速确定子树边界的树结构，如多叉树的序列化与反序列化。

2. 带度数的层次次序

- **原理**：按**层次遍历**（一层一层访问节点）存储时，每个节点附带记录自身的 “度数”。
- **作用**：层次遍历时，若某节点度数为k，则其下一层的连续k个节点即为它的子节点。通过度数可严格遵循层次结构还原树，避免顺序存储的歧义。
- **适用场景**：适用于需要保留层次关系的树结构，如组织结构树、层级菜单的存储与解析。

三、二叉树的顺序存储

1. 二叉树与森林的对应（语义差异）

森林可通过 **“孩子 - 兄弟” 表示法**转换为二叉树（森林中第一棵树的根为二叉树根，第一棵树的子树森林转换为二叉树的左子树，其余树组成的森林转换为二叉树的右子树）。但二者**语义不同 **：森林是 “多棵树的集合”，二叉树是 “单棵树的结构”，仅在存储结构上存在对应关系。

2. 带链的二叉树顺序存储变种

- **带右链的二叉树前序**：前序遍历二叉树时，通过 “右链” 记录节点的**兄弟关系**。在顺序存储中，右链可体现 “同一父节点下的兄弟节点序列”，辅助还原二叉树的兄弟连接

逻辑。

- **带左链的二叉树层次次序**：层次遍历二叉树时，通过 “左链” 记录节点的**子节点关系**。在顺序存储中，左链可体现 “层次结构中父节点到子节点的连接”，辅助还原二叉树的层次父子逻辑。

关于K叉树

注意二叉树是K叉树特例，但K叉树不是树特例（对每个树也存在一个K）

- 注意一下，遍历方式的对应：二叉先序对应树先根；中序对后根
- 树中节点的“度”按定义只看往下的边，上面长出它的不算

Acknowledgement

本文档参考了

王乐业老师的2025fall PPT

2024年及以前的学长学姐整理的资料，包括xjgg的GitHub仓库，+vgg的资料包等，特别予以鸣谢（@ICUlizhi 徐靖学长， @C+V 陈佳伟学长）

（也有很多资料并没有特别标注出处，如有缺漏望海涵）

PKUHUB是我的神😍